

Lista 1 (Lab) Termin **oddania** do 20.10.2024

Uwaga: Zadania z tej listy proszę nie wysyłać na SVN!

1. (2pt) Zainstaluj program do wirtualizacji VirtualBox lub QEMU lub libvirt (można wykorzystać też dowolny inny program do wirtualizacji). Zapoznaj się z programem.
2. (5pt) Zainstaluj w środowisku wirtualnym dystrybucję GNU/Linux np. Ubuntu. Pobierz obraz systemu tutaj. Naucz się podstawowych komend obsługi systemu z konsoli np. `cd`, `ls`, `cat`, `less`, `apt-get` itp.
3. (8pt) Zainstaluj w środowisku wirtualnym dystrybucję GNU/Linux Arch Linux. Pobierz obraz systemu tutaj. Naucz się podstawowych informacji o systemie oraz umiej wytłumaczyć w kilku zdaniach co najmniej
 - o Co to jest BIOS oraz UEFI?
 - o Co to jest GPT, MBR?
 - o Co to jest i jak obsługiwać program do robienia partycji dysku np. fdisk? Wykonaj podstawowe polecenia programu samodzielnie w maszynie wirtualnej
 - o Co to jest system plików i jakie systemy plików wykorzystuje Linux?
 - o Ogólne pytania z procesu instalacji

Przykładowe instalacje Arch Linuksa i instrukcja oraz instalacja w VirtualBox

4. (10pt) W tym zadaniu zainstaluj system xv6, który będziemy wykorzystywać jako referencyjną implementację prostego systemu operacyjnego (wersja dla procesora x86). W środowisku GNU/Linux (Ubuntu, ArchLinux, ...) pobierz z github system operacyjny xv6 oraz pobierz łąkę xv6.patch. Łąka pozwala na kompilację dla nowszych wersji `gcc ≥ 12`:

```
5.
6.$ git clone https://github.com/mit-pdos/xv6-public.git
7.Cloning into 'xv6-public'...
8....
9.$ cd xv6-public
10.    $ wget https://cs.pwr.edu.pl/zawada/so/xv6.patch
11.    $ patch -pl -i xv6.patch
12.
```

Możesz teraz zbudować i uruchomić ten system. Na początek musisz zainstalować emulator qemu (np. `apt install qemu` lub `pacman -S qemu-desktop`):

```
$ make qemu
```

Można też uruchomić bezpośrednio w terminalu (bez środowiska graficznego) przez emulator `qemu` tak:

```
$ make qemu-nox
```

Wyjście z emulatora mamy przez kombinację klawiszy `ctrl+a x`, czyli jednocześnie naciskamy `ctrl` i `a`, potem klawisz `x`. Pomoc skrótów pokazuje kombinacja `ctrl+a h` np. `ctrl+a c` - konsola `qemu` gdzie można zobaczyć dużo informacji o maszynie emulującej przez `qemu` np. `info`, `info mem`, `info block`, `info registers`, `info qtree`. System możemy śledzić przez debugger `gdb` uruchamiając:

```
$ make qemu-gdb # lub make qemu-nox-gdb
```

i w drugim terminalu na początek pozwolmy `gdb` na automatyczne ładowanie pliku `.gdbinit` który jest przygotowany w źródłach `xv6`:

```
$ echo "set auto-load safe-path /" >> ~/.gdbinit # komendę uruchamiamy tylko raz!
```

Uruchamiamy `gdb`:

```
$ cd xv6-public
$ gdb ./kernel
GNU gdb (GDB) 15.1
:
Reading symbols from ./kernel...
+ target remote localhost:26000
The target architecture is set to "i8086".
[f000:fff0] 0xffff0: ljmp $0x3630,$0xf000e05b
0x0000fff0 in ?? ()
+ symbol-file kernel
```

```
(gdb) break *0x7c00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.
[ 0:7c00] => 0x7c00:cli

Thread 1 hit Breakpoint 1, 0x00007c00 in ?? ()
(gdb)
```

Szczegóły tego, co widzisz, prawdopodobnie będą się różnić od powyższych w zależności od wersji gdb, której używasz, ale gdb powinien się zatrzymać w punkcie przerwania i powinna to być powyższa instrukcja cli.

13. (15pt)* Zainstaluj w środowisku wirtualnym system operacyjny z rodziny BSD FreeBSD. Jako podstawowy system plików wybierz ZFS oraz wytłumacz co to są jails oraz pokaż przykład zastosowania. Umiej zastosować wirtualizację z wykorzystaniem bhyve.

Lista 2 (Lab) Termin wysłania na SVN do 3.11.2024

Na SVN proszę wysłać tylko zadania 6, 7, 8 i 9. Resztę zadań należy oddać najpóźniej na pierwszych (lub drugich) zajęciach po terminie 4.11.2024.

1. (5pt) Wytłumacz jakie pliki zawierają katalogi `/dev`, `/proc` oraz `/sys`. Wykorzystując polecenie `dd` odczytaj pierwszy sektor z dysku głównego (uwaga na prawa dostępu) lub podpiętego pendrive'a i wyświetl przez `hexdump -C`. Z katalogu `proc` wyświetl informacje o pamięci, procesorze i partycjach.
2. (5pt) Zapoznaj się z programem `ps` (`man ps`). Naucz się posługiwać tym programem, aby umieć sprawdzać co najmniej istnienie i parametry wybranych procesów (PID procesu i rodzica, stan procesu, priorytet, nice, ilość pamięci, zużycie czasu procesora). Uruchom też kilka terminali pokaż jakie urządzenie tty wykorzystują. Wykonując komendę `ps axjf` pokaż wszystkie procesy które podpięte są do tych terminali (kolumna TTY).
3. (5pt) Zapoznaj się z kompilatorem języka C (polecenie `gcc`) oraz języka C++ (polecenie `g++`). Uruchom poniższy program w języku C.

```
4.
5.$ cat > test.c
6.#include <stdio.h>
7.
8.int main(int argc, char *argv[])
9.{
10.    printf("Hello, World!\n");
11.    return 0;
12.}
13.^D
14.$ gcc -Wall -pedantic test.c
15.$ ./a.out
16.
```

Wytłumacz każdy z powyższych kroków. Co oznaczają opcje `-Wall` oraz `-pedantic`? Zobacz `man gcc`.

17. (5pt) Pokaż na przykładzie (np. `sleep 1000`, `sleep 1001`, ...) zarządzanie zadaniami wykorzystując `&` - uruchamianie w tle (background) oraz `jobs`, `fg`, `bg`, `kill` oraz `^Z`. Uruchom kilka zadań w tle i pokaż jak można nimi zarządzać, czyli zatrzymywać, wznowiać oraz kończyć ich działanie. Pokaż jak uruchomione zadanie (nie w tle), można w czasie działania uruchomić w tle np. wykonując komendę `sleep 100` (bez `&`) w czasie działania przełącz je do działania w tle.
18. (5pt) Poleceniem `mkfifo` (`man mkfifo`) utwórz potok nazwany (ang. *named FIFO*). Wywołując polecenie `cat` w różnych terminalach spowoduj wpisywanie danych do potoku przez jeden(ne) proces(y), i odczytywanie i wyświetlanie ich przez inne. Zaobserwuj, kiedy polecenie `cat` czytające z potoku czeka na więcej danych, a kiedy kończy pracę.

Analogicznie, kiedy czeka na więcej danych (z klawiatury) polecenie **cat** piszące do potoku?

19. (8pt) Napisz program w języku C, który wykorzystując sekwencje Esc (ang. escape sequences) standardu ANSI wyświetli na ekranie napis "Hello, World!", po kolei we wszystkich dostępnych przez terminal kolorach. Czy terminal może wyświetlić 256 lub więcej kolorów?
20. (10pt) Napisz potok poleceń (może być w skrypcie), który zamienia wszystkie nazwy plików w danym katalogu (bez podkatalogów) na małe litery, czyli wszystkie duże litery występujące w nazwach plików zostaną zamienione na małe, a małe litery pozostają oczywiście dalej małe. Skrypt powinien działać poprawnie na takich nazwach plików jak " ABC DEF", "-AbC aBc" i "--ABC DEF". Przy oddawaniu, proszę podane nazwy plików zakładać w konsoli korzystając z polecenia **touch**.
21. (10pt) Pobierz z github program hello-os następnie zastosuj łatę patch która modyfikuje program i pozwala na pisanie programu w języku C (zamiast w asm). Zmodyfikuj program w pliku **boot.c** tak aby wyświetlił napis "Hello World!" we wszystkich dostępnych kolorach (każdy napis w osobnej linii) tła oraz kolorach czcionki. Program uruchamiamy przez **make qemu**. Można również uruchomić program tylko w terminalu zmieniając **-vga std** na **-display curses** w pliku **Makefile** np. wykonując w terminalu

```
22.  
23.      $ sed -i 's/vga std/display curses/g' Makefile  
24.
```

Po uruchomieniu **make qemu** qemu uruchomi się tylko w terminalu, przez **Alt+2** możemy przejść do monitora qemu i np. wyjść pisząc **quit** resztę komend możemy zobaczyć pisząc **help**. Jeśli chcemy wrócić do emulacji to wciskamy **Alt+1**.

25. (15pt)* W shellu można przekierować standardowe wejście, wyjście i wyjście błędu przez wykorzystanie przekierowań **<**, **>** oraz **2>**. Założmy jednak że już wykonaliśmy komendę i dopiero w czasie działającego już procesu chcemy zmienić wejście, wyjście lub wyjście błędu. Na przykład

```
26.  
27.      $ sleep 1000 &  
28.      [1] 4096  
29.      $ ls -l /proc/4096/fd  
30.      lrwx----- 1 user users 64 10-30 12:32 0 -> /dev/pts/4  
31.      lrwx----- 1 user users 64 10-30 12:32 1 -> /dev/pts/4  
32.      lrwx----- 1 user users 64 10-30 12:32 2 -> /dev/pts/4  
33.      $ # tutaj zmieniamy wyjście działającego procesu 4096 na plik /tmp/foo  
34.      $ # oczywiście nie kończymy działa procesu np. przez kill 4096 w  
    trakcie  
35.      $ # zmiany wyjścia ...  
36.      $ ...  
37.      $  
38.      $ ls -l /proc/4096/fd  
39.      lrwx----- 1 user users 64 10-30 12:32 0 -> /dev/pts/4  
40.      lrwx----- 1 user users 64 10-30 12:32 1 -> /tmp/foo  
41.      lrwx----- 1 user users 64 10-30 12:32 2 -> /dev/pts/4  
42.
```

Napisz program który dokonuje takiej zmiany. Tutaj nie korzystamy z takich narzędzi jak gdb lub inny debugger! Wykorzystaj do tego ptrace. Dobre wprowadzenie do ptrace można znaleźć w tych dwóch artykułach: Playing with ptrace, Part I, Playing with ptrace, Part II.

Przykład konwersji stron podręcznika **man** do formatu PDF:

```
$ man -t ps | ps2pdf - ps.pdf  
$ man -t gcc | ps2pdf - gcc.pdf
```

W przypadku braku programu **ps2pdf**, należy zainstalować pakiet **ghostscript** np. pod archlinuksem:

```
# pacman -S ghostscript
```

Lista 3 (Lab) Termin wysłania na SVN do 17.11.2024

1. (10pt) Napisz skrypt w Bashu, który pokazuje informacje o wszystkich procesach (podobne jak program **ps aux**). W zadaniu wykorzystaj system plików **proc** (standardowo w systemie Linux montowanym w katalogu **/proc**, **man 5 proc**) do pobrania informacji o procesach np. **cat /proc/PID/status** (i/lub **/proc/PID/stat**) wyświetla informacje o procesie PID. Wyświetl **ppid**, **pid**, **comm**, **state**, **tty**, **rss**, **pgid**, **sid** i poznaj dokładnie wszystkie wymienione oznaczenia! Dodatkowo wyświetl informację ile proces ma otwartych plików. Wszystkie te informacje o jednym procesie mają być "ładnie" (patrz np. **man column**) wyświetlone w jednym wierszu podobnie jak w programie **ps**.
2. (15pt) Napisz skrypt w Bashu, który co sekundę prezentuje informacje o systemie operacyjnym. Dane pobierz wykorzystując pseudo system plików **proc** w Linuksie domyślnie zamontowanym w katalogu **/proc** (patrz **man 5 proc**) oraz **sysfs** (patrz **man 5 sysfs**). Skrypt powinien prezentować następujące informacje:
 - o Aktualną oraz średnią prędkość wysyłania i odbierania danych z sieci (odczytaj i zinterpretuj **/proc/net/dev** oraz wyświetl w B, KB lub MB w zależności od aktualnej prędkości) i
 - o Aktualne wykorzystanie rdzeni procesora dla każdego rdzenia osobno w procentach (patrz **/proc/stat** - **man 5 proc**) wraz z aktualną częstotliwością (patrz np. **/sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq** dla **cpu0**) pracy rdzenia procesora (podobnie jak **htop**) i
 - o Jak długo system jest uruchomiony w dniach, godzinach, minutach i sekundach (**/proc/uptime**) i/lub
 - o Aktualny stan baterii w procentach (**/sys/class/power_supply/BAT0/uevent**) i
 - o Obciążenie systemu **/proc/loadavg** oraz
 - o Aktualne wykorzystanie pamięci **/proc/meminfo** (przeanalizuj co oznaczają 3 początkowe wiersze).

W przypadku prędkości przesyłania danych skrypt **powinien** prezentować "graficznie" historię poprzednich pomiarów np. prosty wykres słupkowy. Przykładowe programy z wykresami na których można się wzorować to: **s-tui** lub **bmon** lub **bpytop**. Można wykorzystać inne znaki w UTF-8. Zobacz też informacje o komendzie **tput** np. [link](#) i/lub [link](#). Układ oraz kolejność wyświetlanych informacji jest dowolna.
3. (10pt) Napisz skrypt w Bashu wykorzystujący **REST API** do pobierania danych z dwóch przykładowych źródeł np. **Chuck Norris API** i **The Cat API** lub dowolne inne które pobiera tekst i obrazki. Do zapytań RESTowych wykorzystaj **curl** lub **wget** do parsowania JSONa wystarczy program **jq** (**pacman -S jq**), dla osób korzystających z formatu XML dostępny jest program **xmllint** (**pacman -S libxml2**). Po uruchomieniu skryptu na ekranie pokaż obraz z bazy 'The Cat API' wykorzystując np. program **img2txt** (**pacman -S libcac**) lub **cating** oraz poniżej wyświetl losowy cytat z bazy Chucka Norrisa.
4. (10pt) Napisz program w języku C dla systemu Xv6, który pozwoli na wykonywanie prostych operacji arytmetycznych na liczbach całkowitych w konsoli nazwijmy go **expr**. Program dostając na wejście jako argumenty liczbę, operator i liczbę powinien wracać wynik tej operacji np.

```
5.
6.cpu0: starting 0
7.sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap
  start 58
8.init: starting sh
9.$ expr 2 + 2
10.      4
11.      $ expr 10 / 3
12.      3
13.      $ expr 10 - 2
14.      8
15.
```

Uwaga: Program dołącz do systemu przez prostą modyfikację 'Makefile'.

16. (10pt)* Zmodyfikuj system operacyjny Xv6 (patrz m.in. na **console.c**) tak aby wyświetlać tekst w dowolnych kolorach. Możesz to zrobić podobnie wprowadzając prostą (wystarczy tylko do koloru) ANSI escape sequences lub wykorzystując np. '%' (który już jest), wtedy %1 - kolor niebieski, %2 - kolor czerwony itp. Napisz krótki program testowy **hello** wyświetlający 'Hello, World' we wszystkich kolorach dla Xv6.
17. (10pt)* Celem tego zadania jest dokładniejsze poznanie **ANSI escape sequences** i obsługi terminali. Bazując np. na grze **arkanoid** napisz w Bashu prostą wersję gry która pozwala na ucieczkę z labiryntu. Do generowania labiryntu wykorzystaj dowolne **algorytmy**. Zrób tak aby gra (labirynt) dostosowywała się do wielkości terminala.

Lista 4 (Lab) Termin wysłania na SVN do 1.12.2024

- (5pt) [Linux] Napisz program w języku C, który uruchomi powłokę (Bash) z prawami roota. Po kompilacji programu można ustawić (z poziomu roota) dowolne atrybuty (np. patrz [SUID](#)). Następnie już z poziomu dowolnego użytkownika uruchamiając program uruchamia się konsola administratora, podobnie jak `sudo /bin/bash` (bez wprowadzania hasła). Oczywiście proszę nie wykorzystywać programu 'sudo' we własnym programie!
- (5pt) [Linux i XV6] Napisz w języku C programy testujące, które odpowiedzą na następujące pytania:
 - Czy można napisać program do obsługi wszystkich sygnałów (patrz `kill -1`)? Napisz program prezentujący odpowiedź.
 - Czy jest możliwe wysłać sygnał `SIGKILL`, lub inny do procesu `init` (PID 1) czyli np. `kill -9 1` (nawet będąc `rootem`)?
 - Czy sygnały są kolejkowane? Np. napisz program testowy wysyłający wiele razy do danego procesu sygnał (np. `SIGUSR1`) i zobacz czy wszystkie dotarły.
- (10pt) [Linux] Zaimplementuj w języku C prostą wersję powłoki o nazwie `lsh`. Jak prawdziwa powłoka, `lsh` odczytuje linię ze standardowego wejścia i przeszukuje ścieżki ze zmiennej `PATH` (inaczej mówiąc zamiast `execve` wykonuje `execvp`) i wykonuje podany program. Proszę pamiętać o ustawieniu argumentów wykonywanej komendy. Jeśli linia kończy się znakiem (&), wtedy `lsh` powinien nie czekać aż komenda zostanie skończona i od razu wrócić. W innym przypadku `lsh` powinien zaczekać, aż program wykona się. `lsh` powinien skończyć swoje działanie naciskając klawisze `Control+D` lub pisząc `exit`. Zmieniamy katalogi przez wpisanie komendy `cd`. Komendy `cd` oraz `exit` to komendy wbudowane. **Uwaga:** Procesy uruchomione w tle, które się zakończyły mogą stać się procesami 'zombi', rozwiąż ten problem w `lsh`. Znajdziesz w sieci wiele rozwiązań np. [Build your own shell](#) lub `sh` z `xv6`, ale to ma być wersja minimalna wystarczy napisać prosty parser dla linii komend oraz zastosować kilka wywołań systemowych!
- (10pt) [Linux] Zaimplementuj w programie `lsh` z poprzedniego zadania potoki | (ang. pipe) oraz przekierowanie standardowego wejścia (<), wyjścia (>) oraz wyjścia błęd (2>). Wskazówka: Zobacz program [lssort.c](#). Ponadto Ctrl-C przerywa wykonywanie programu w powłoce (nie samej powłoki oraz zadań wykonywanych w tle).
- (15pt)* [Linux] Zaimplementuj w programie `lsh` zarządzanie zadaniami ([job-control](#)), czyli co najmniej komendy wbudowane `fg`, `bg`, `jobs` oraz `^Z`. Patrz książka [Michael Kerrisk, "Linux Programming Interface"](#).
- (15pt)* [XV6] Dopisz do `xv6` nowe wywołania systemowe. Jako pierwsze napisz wywołanie systemowe o nazwie `hello` wyświetla tylko napis "Hello World!" oraz wywołanie systemowe `getppid` które wraca pid rodzica. Napisz program w języku C, który testuje nowe wywołania systemowe. **Uwaga:** Pamiętaj, że dodanie nowego wywołania, aby rozszerzyć funkcjonalność jądra nie powinno być stosowane jako pierwszy wybór! Potraktuj to zadanie tylko jako ćwiczenie.

Lista 5 (Lab) Termin wysłania na SVN do 15.12.2024

- (5pt) Napisz program w asemblerze NASM (lub innym) dla procesora x86, który:
 - Pobiera liczbę całkowitą w postaci tekstowej z wejścia standardowego (np. "12345")
 - Oblicza sumę cyfr tej liczby (w przykładzie: $1 + 2 + 3 + 4 + 5 = 15$)
 - Wyświetla wynik na standardowym wyjściuDo wczytywania i wyświetlania użyj wywołań systemowych `read` i `write`.
- (5pt) Napisz program w asemblerze NASM (lub innym) dla procesora x86, który:
 - Przyjmuje jako dane wejściowe macierz 3x3 (9 liczb całkowitych, zapisanych w pamięci programu).
 - Wylicza sumę wszystkich elementów macierzy oraz sumę elementów na przekątnej głównej.

- Wyświetla oba wyniki na standardowym wyjściu
- Suma elementów: X
- Suma przekątnej: Y

Użyj wywołań systemowych do wyświetlania wyników. Wyjaśnij w gdb, jak program przechowuje macierz w pamięci i jak przetwarza jej elementy.

3. (10pt) Napisz program w asemblerze NASM (lub innym) dla procesora x86, który wyświetla na standardowym wyjściu liczbę 32-bitową w kodzie szesnastkowym, czyli np. takie `printf("%x\n",123456)`. Do wyświetlania wykorzystaj wywołanie systemowe `write`. Pokaż jak program wykonuje się w gdb (debugger).
4. (10pt) Napisz program w asemblerze x86 (32-bit), który wysyła na standardowe wyjście liczby pierwsze z zakresu od 1 do 100 000.
5. (10pt) Napisz w asemblerze x86 program rysujący zbiór Mandelbrota (patrz wykład 2024.12.02). Inicjalizację trybu graficznego można przeprowadzić wykorzystując kod boot (przy oddawaniu należy umieć wytłumaczyć kod `boot.asm`).
6. (15pt*) Napisz program w asemblerze NASM (lub innym) dla procesora x86 z rozkazami SIMD (SSE2, SSE4, AVX, ...), który liczy silnię $n!$ na liczbach 512 bitowych. Zauważ, że np. 128-bitowe rejestry XMM0,...,XMM7 od wersji SSE2, mogą być traktowane jako dwa 64-bitowe liczby całkowite lub cztery 32-bitowe itd. Liczba n pobierana jest z argumentów `argc`, `argv` (lub ze standardowego wejścia) oraz wynik wysyłany jest na standardowe wyjście.

Przydatne linki:

- [Learn Assembly Language](#)
- [PC Assembly Language, Paul A. Carter, November 2003](#)
- [Intel 80386 Programmer's Reference Manual, 1987](#)
- [x86-64 Assembly Language Programming with Ubuntu, 2022](#)

Lista 6 (Lab) Termin wysłania na SVN do 12.01.2025

Na SVN proszę wysłać tylko różnice (diff) np.

```
$ git clone https://github.com/mit-pdos/xv6-public.git
```

```
$ cd xv6-public
```

```
$ # TUTAJ PROSZE PRZEGRĄĆ WSZYSTKIE ZMODYFIKOWANE PLIKI
```

```
$ git diff > listaX_zadX.patch
```

lub jeśli dodatkowo dodaliśmy nowe pliki (np. `testvm.c`)

```
$ git clone https://github.com/mit-pdos/xv6-public.git
```

```
$ cd xv6-public
```

```
$ # TUTAJ PROSZE PRZEGRĄĆ WSZYSTKIE ZMODYFIKOWANE PLIKI
```

```
$ git add "TUTAJ PLIKI np. testvm.c"
```

```
$ git commit -a -m "listaX_zadX"
```

```
$ git format-patch -1
```

Na SVN proszę przesłać tylko lista6_zad1.patch i lista6_zad2.patch

1. (10pt) Zaimplementuj w XV6 funkcję **vmprint**, która drukuje zawartość tabeli stronicowania procesu (pgdir). Zdefiniuj funkcję w **kernel/vm.c**. Wstaw funkcję **vmprint** do wywołania **exec.c**, aby wydrukować tabelę stron dla aktualnie uruchomionego procesu użytkownika oraz wstaw **vmprint** w **exit (proc.c)**. Następnie napisz program **testvm.c** który wykorzystując wywołanie **sbrk** zaalokuje 5 stron pamięci. Przykładowe dane wyjściowe wydrukowane przez **vmprint** dla uruchomiana programu **testvm** oraz **ls** mogą wyglądać następująco:

2.init: starting sh

3.exec - pgdir 8df73000

4... 0: pde 0xdf31000 pa 0x8df31000

5... .. 0: pte 0xdf32000 pa 0x8df32000

6... .. 1: pte 0xdf30000 pa 0x8df30000

7... .. 2: pte 0xdf2f000 pa 0x8df2f000

8... .. 3: pte 0xdf2e000 pa 0x8df2e000

9.\$ testvm

10.exec - pgdir 8df23000

11... 0: pde 0xdee1000 pa 0x8dee1000

12... .. 0: pte 0xdee2000 pa 0x8dee2000

13... .. 1: pte 0xdee0000 pa 0x8dee0000

14... .. 2: pte 0xdedf000 pa 0x8dedf000

15.exit - pgdir 8df23000

16... 0: pde 0xdee1000 pa 0x8dee1000

17... .. 0: pte 0xdee2000 pa 0x8dee2000

18... .. 1: pte 0xdee0000 pa 0x8dee0000

19... .. 2: pte 0xdedf000 pa 0x8dedf000

20... .. 3: pte 0xdf2d000 pa 0x8df2d000

21... .. 4: pte 0xdfbd000 pa 0x8dfbd000

22... .. 5: pte 0xdf74000 pa 0x8df74000

23... .. 6: pte 0xdede000 pa 0x8dede000

24... .. 7: pte 0xdedd000 pa 0x8dedd000

25.\$ ls

26.exec - pgdir 8dfffb000

```

27... 0: pde 0xdf28000 pa 0x8df28000

28... .. 0: pte 0xdf29000 pa 0x8df29000

29... .. 1: pte 0xdf27000 pa 0x8df27000

30... .. 2: pte 0xdf26000 pa 0x8df26000

31..          1 1 512

32...          1 1 512

33.README      2 2 2286

34.cat          2 3 15832

35.echo         2 4 14736

36.forktest     2 5 8764

37.grep         2 6 18752

38.init         2 7 15332

39.kill         2 8 14804

40.ln           2 9 14676

41.ls           2 10 17316

42.mkdir        2 11 14800

43.rm           2 12 14780

44.sh           2 13 28892

45.stressfs     2 14 15628

46.usertests    2 15 63588

47.wc           2 16 16188

48.zombie       2 17 14420

49.console      3 18 0

50.exit - pgdir 8dfffb000

51... 0: pde 0xdf28000 pa 0x8df28000

52... .. 0: pte 0xdf29000 pa 0x8df29000

53... .. 1: pte 0xdf27000 pa 0x8df27000

54... .. 2: pte 0xdf26000 pa 0x8df26000

```

- Pierwszy wiersz drukuje adres argumentu `vmprint`.

- Każdy wiersz "pde" pokazuje indeks PDE w jego katalogu stron, "pa", adres fizyczny dla PDE.
- Wyjście powinno również wskazywać poziom katalogu stron: wpisy najwyższego poziomu są poprzedzone przez "..", następny poziom niżej innym ".."
- Nie należy drukować wpisów, które nie są zamapowane oraz PDE sprawdzamy tylko do 512 (dalej jest mapowanie kernela). PTE normalnie do 1024.
- W powyższym przykładzie dla programu `ls` katalog stron najwyższego poziomu ma mapowania dla wpisu 0. Następny poziom niżej dla wpisu 0 ma zamapowane wpisy 0, 1 i 2.
-

55. (20pt) Napisz w Xv6 wywołanie systemowe, które wyświetlania informacje o pamięci. Zaimplementuj:

- **usedvp()** powinno zwrócić liczbę stron wirtualnych/logicznych w części użytkownika przestrzeni adresowej procesu, do rozmiaru programu zapisanego w `struct proc`. Musisz również uwzględnić stronę ochrony stosu w swoich obliczeniach.
- **usedpp()** powinno zwrócić liczbę stron fizycznych w części użytkownika przestrzeni adresowej procesu. Musisz policzyć tę liczbę, przechodząc przez tabelę stron procesu i licząc liczbę wpisów w tabeli stron, które mają przypisany prawidłowy adres fizyczny.

Ponieważ xv6 nie używa stronicowania na żądanie (paging on demand), możesz oczekiwać, że liczba stron wirtualnych i fizycznych będzie taka sama (sytuacja ulegnie zmianie w zadaniu 3).

Wskazówka: możesz przejść przez tabelę stron procesu, używając funkcji `walkpgdir`, która jest obecna w `vm.c`. Możesz znaleźć kilka przykładów, jak wywołać tę funkcję w samym `vm.c`. Aby obliczyć liczbę stron fizycznych w procesie, możesz napisać funkcję, która przegląda tabelę stron procesu w pliku `vm.c` i wywołać tę funkcję z kodu obsługi wywołań systemowych. Napisz program testujący np:

```
int main() {

    int used_virtual = usedvp();

    int used_physical = usedpp();


    printf(1, "Liczba użytych stron wirtualnych: %d\n", used_virtual);

    printf(1, "Liczba użytych stron fizycznych: %d\n", used_physical);


    // Przydziel pamięć, aby sprawdzić, czy liczniki się zmieniają

    sbrk(4096);

    used_virtual = usedvp();

    used_physical = usedpp();


    printf(1, "Po przydzieleniu jednej strony:\n");

    printf(1, "Liczba użytych stron wirtualnych: %d\n", used_virtual);

    printf(1, "Liczba użytych stron fizycznych: %d\n", used_physical);
```

```

    exit();
}

```

56. (10pt)* Napisz w XV6 prostą wersję mapowania pamięci za pomocą wywołania systemowego `mmap`. To wywołanie systemowe `mmap` powinno przyjmować jeden argument: liczbę bajtów do dodania do przestrzeni adresowej procesu. Można założyć, że liczba bajtów jest liczbą dodatnią i wielokrotnością rozmiaru strony. Wywołanie systemowe powinno zwrócić wartość 0, jeśli podano jakiekolwiek nieprawidłowe dane wejściowe. Jeśli podano prawidłową liczbę bajtów jako dane wejściowe, wywołanie systemowe powinno rozszerzyć wirtualną przestrzeń adresową procesu o określoną liczbę bajtów i zwrócić początkowy adres wirtualny nowo dodanego obszaru pamięci. Nowe wirtualne strony powinny zostać dodane na końcu bieżącego końca programu i powinno odpowiednio zwiększyć rozmiar programu. Jednak wywołanie systemowe NIE powinno przydzielać żadnej pamięci fizycznej odpowiadającej nowym stronom wirtualnym, ponieważ będziemy przydzielać pamięć na żądanie (on demand) w zadaniu następnym. Możesz użyć wywołania systemowego z poprzedniego zadania, aby wydrukować te liczby stron w celu zweryfikowania implementacji. Po wywołaniu systemowym `mmap` i przed, powinieneś zobaczyć tylko wzrost liczby stron wirtualnych procesu, ale nie liczbę stron fizycznych. Pamięć fizyczna dla mapowanej w pamięci strony wirtualnej powinna być przydzielana na żądanie, tylko wtedy, gdy użytkownik uzyskuje dostęp do strony. Gdy użytkownik uzyskuje dostęp do mapowanej w pamięci strony, wystąpi błąd strony (page fault). W tym zadaniu po prostu dostajemy błąd. Napisz prosty program testowy do przetestowania implementacji. Ten program wywołuje `mmap` wiele razy. Drukuje on liczbę stron wirtualnych i fizycznych okresowo, aby sprawdzić, czy liczba stron jest poprawnie aktualizowana. Możesz napisać więcej takich przypadków testowych, aby dokładnie przetestować implementację.
57. (20pt)* W tym zadaniu dalej rozszerzamy zadanie poprzednie na pełne działanie `mmap`. Na początku musimy przydzielać pamięć fizyczną w ramach obsługi błędu strony. Ponadto, jeśli mapujesz w pamięci więcej niż jedną stronę, pamięć fizyczna powinna być przydzielana tylko dla tych stron, do których uzyskuje się dostęp, a nie dla wszystkich stron w regionie mapowanym w pamięci. Ponownie użyj liczby stron wirtualnych/fizycznych, aby sprawdzić, czy strony fizyczne są przydzielane tylko na żądanie. Napisz prosty program testowy do przetestowania implementacji. Ten program powinien wywoływać `mmap` wiele razy i **uzyskuje dostęp** do mapowanych w pamięci stron. Drukuje on liczbę stron wirtualnych i fizycznych okresowo, aby umożliwić sprawdzenie, czy liczba stron jest poprawnie aktualizowana. Możesz napisać więcej takich przypadków testowych, aby dokładnie przetestować implementację.

Wskazówki:

- Przeanalizuj implementację wywołania systemowego `sbrk`. Wywołanie systemowe `mmap` będzie przebiegać według podobnej logiki. W `sbrk` przestrzeń adresów wirtualnych jest zwiększana, a pamięć fizyczna jest przydzielana w ramach tego samego wywołania systemowego. Implementacja `sbrk` wywołuje funkcję `growproc`, która z kolei wywołuje funkcję `allocuvmm` w celu przydzielenia pamięci fizycznej. W przypadku implementacji `mmap` musisz zwiększyć przestrzeń adresów wirtualnych tylko w ramach implementacji wywołania systemowego, a pamięć fizyczna musi zostać przydzielona podczas błędu strony. Możesz wywołać `allocuvmm` (lub napisać inną podobną funkcję), aby przydzielić pamięć fizyczną w przypadku błędu strony.
- Oryginalna wersja xv6 nie obsługuje pułapki błędu strony (**page fault trap**). W przypadku tego zadania należy napisać dodatkowy kod do obsługi pułapki błędu strony w `trap.c`, który przydzieli pamięć na żądanie dla strony, która spowodowała błąd strony. Możesz sprawdzić, czy pułapka jest błędem strony, sprawdzając, czy `tf->trapno` jest równe `T_PGFLT`. Przyjrzyj się argumentom instrukcji `cprintf` w `trap.c`, aby dowiedzieć się, jak znaleźć adres wirtualny, który spowodował błąd strony. Użyj `PGROUNDDOWN(va)`, aby zaokrąglić adres wirtualny powodujący błąd w dół do początku granicy strony. Po napisaniu kodu do obsługi błędu strony wykonaj `break` lub `return`, aby uniknąć przetwarzania innych pułapek (traps).
- Pamiętaj: ważne jest wywołanie `switchvmm` w celu aktualizacji rejestru `CR3` i `TLB` za każdym razem, gdy zmieniasz tabelę stron procesu. Ta

aktualizacja tabeli stron umożliwi procesowi wznowić wykonywanie, gdy poprawnie obsłużysz błąd strony.

Przydatne linki:

- o [książka x86](#)
- o [kod źródłowy projektu \(x86\)](#)
- o [książka z kodem źródłowym \(x86\)](#)

Lista 7 (Lab) Termin wysłania na **SVN** oraz oddania do **4.02.2025**

Na SVN proszę wysłać tylko różnice (diff) np.

```
$ git clone https://github.com/mit-pdos/xv6-public.git
```

```
$ cd xv6-public
```

```
$ # TUTAJ PROSZE PRZEGRAĆ WSZYSTKIE ZMODYFIKOWANE PLIKI
```

```
$ git diff > listaX_zadX.patch
```

lub jeśli dodatkowo dodaliśmy nowe pliki (np. test.c)

```
$ git clone https://github.com/mit-pdos/xv6-public.git
```

```
$ cd xv6-public
```

```
$ # TUTAJ PROSZE PRZEGRAĆ WSZYSTKIE ZMODYFIKOWANE PLIKI
```

```
$ git add "TUTAJ PLIKI np. test.c"
```

```
$ git commit -a -m "listaX_zadX"
```

```
$ git format-patch -1
```

Na SVN proszę przesłać tylko lista7_zad1.patch, lista7_zad2.patch, ...

1. (10pt) Napiszmy nowe wywołanie systemowe w xv6 które wraca informacje o procesach w systemie oraz program **ps**, który wykorzystując to wywołanie wyświetla wszystkie aktywne procesy. Zaimplementuj wywołanie systemowe **getNumProc()**, aby zwrócić całkowitą liczbę aktywnych procesów w systemie (w stanie embrionalnym (EMBRYO), działającym (RUNNING), możliwym do uruchomienia (RUNNABLE), uśpionym (SLEEPING) lub zombie (ZOMBIE). Zaimplementuj również wywołanie systemowe **getMaxPid()**, które zwraca maksymalny **PID** spośród **PID** wszystkich aktualnie aktywnych procesów w systemie. Następnie zaimplementuj wywołanie systemowe **getProcInfo(pid, &processInfo)**. To wywołanie systemowe przyjmuje jako argumenty całkowity **PID** i wskaźnik do struktury **processInfo**. Ta struktura służy do przekazywania informacji między trybem użytkownika a trybem jądra. Informacje o procesie, które muszą zostać zwrócone, obejmują nadrzędny **PPID**, liczbę przełączeń kontekstu procesu przez planistę (scheduler) oraz rozmiar procesu w bajtach. Należy zauważyć, że chociaż część tych informacji jest już dostępna jako część struktury **proc** procesu, należy dodać nowe pola, aby śledzić inne dodatkowe informacje. Należy zauważyć, że **PPID** rodzica procesu **init** (**PID=1**) jest ustawiony na **0** zgodnie z konwencją. Po połączeniu wszystkich tych wywołań systemowych możliwe jest iterowanie po wszystkich aktywnych

procesach w systemie i wyświetlanie ich informacji na ekranie, tak jak polecenie **ps** w systemie Linux. W przypadku wszystkich wywołań systemowych, które nie mają jawnej wartości zwracanej wymienionej powyżej, musisz zwrócić 0 w przypadku sukcesu i wartość ujemną w przypadku niepowodzenia. **Uwaga:** Ważne jest, aby pamiętać, że struktura tabeli procesów **ptable** jest chroniona blokadą. Musisz uzyskać blokadę przed uzyskaniem dostępu do tej struktury w celu odczytu lub zapisu i musisz ją zwolnić po zakończeniu.

2. (15pt) Obecny planista (scheduler) w xv6 to prosty algorytm round robin. Zmodyfikujemy ten scheduler, aby uwzględnić zdefiniowane przez użytkownika priorytety procesów i zaimplementować planistę z priorytetami. Najpierw dodaj nowe wywołanie systemowe do xv6, aby ustawić/pobrać priorytety procesów. Gdy proces wywołuje **setprio(n)**, priorytet procesu powinien zostać ustawiony na określoną wartość. Priorytet może być dowolną dodatnią wartością całkowitą, przy czym wyższe wartości oznaczają wyższy priorytet. Dodaj również wywołanie systemowe **getprio()**, aby odczytać priorytety. Możesz założyć maksymalną wartość priorytetu (powiedzmy 1000), aby uprościć implementację. Następnie zmodyfikuj planistę xv6, aby użyć tego priorytetu przy wyborze następnego procesu do uruchomienia, używając priorytetów jako wag, aby zaimplementować ważony round robin. Należy osiągnąć dwie rzeczy: (a) wyższy priorytet liczbowy powinien przełożyć się na więcej czasu procesora dla procesu, tak aby procesy o wyższym priorytecie kończyły się szybciej niż procesy o niższym priorytecie, oraz (b) procesy o niższym priorytecie nie powinny być nadmiernie głodzone i powinny uzyskać trochę czasu procesora nawet w obecności procesów o wyższym priorytecie. Upewnij się, że poprawnie obsługujesz wszystkie przypadki skrajne w implementacji. Na przykład, możesz musieć ustawić domyślny priorytet dla nowych procesów. Upewnij się również, że Twój kod jest bezpieczny, gdy jest uruchamiany na wielu rdzeniach procesora, używając blokad podczas uzyskiwania dostępu do struktur danych jądra. Musisz pomyśleć o tym, jak będziesz testować poprawność swojego planisty. Wymyśl przypadki testowe, które pokażą Twoją nową politykę planisty. Jeśli planista działa prawidłowo, to procesy o wyższym priorytecie zakończą zadanie przed procesami o niższym priorytecie, ponieważ otrzymają więcej czasu na uruchomienie.
3. (15pt) W tej części zaimplementujemy prostą dodatek do wywołania systemowego **fork**, aby zrozumieć, jak procesy wracają z **trap**. Domyślne zachowanie wywołania systemowego **fork** polega na tym, że sklonowany proces potomny rozpoczyna wykonywanie od tego samego punktu w kodzie, do którego rodzic powraca po wywołaniu systemowym **fork**. W tej części zmodyfikujesz zachowanie **fork**, aby umożliwić procesowi potomnemu wznowienie wykonywania, najpierw w zarejestrowanej funkcji ustawionej przez rodzica, a następnie powrót do kodu po **fork**. Ta funkcjonalność zostanie zaimplementowana za pomocą następujących dwóch nowych wywołań systemowych.
 - o Proces nadrzędny, który chce, aby proces potomny rozpoczął wykonywanie w innej funkcji, powinien wywołać wywołanie systemowe **forkcb** (fork callback). To wywołanie systemowe przyjmuje adres funkcji jako argument. Jeśli ta funkcja nie jest ustawiona, procesy potomne powinny rozpocząć wykonywanie zaraz po **fork**, jak zwykle. Jeśli adres funkcji jest zarejestrowany za pomocą tego wywołania systemowego, nowe procesy powinny rozpocząć wykonywanie w tej funkcji.
 - o Dziecko, które rozpoczyna wykonywanie w funkcji ustawionej przez rodzica, musi wywołać wywołanie systemowe **exitcb** (exit callback), aby powrócić do wykonywania kodu po wywołaniu systemowym **fork**, tak jak robią to zwykle procesy potomne. To wywołanie systemowe nie przyjmuje żadnych argumentów i zostanie wywołane przez dziecko na końcu wykonywania przez nie funkcji.Implementacja tej funkcjonalności będzie wymagała odpowiedniej modyfikacji ramki pułapki (trapframe) procesu potomnego, aby zmienić **EIP** tak, aby wskazywał początkowo na zarejestrowaną funkcję i przywrócić ją do jej pierwotnej wartości, gdy dziecko zakończy wykonywanie funkcji. Poniżej znajduje się przykładowy program testowy:

```
void testfunc(void) {  
  
    printf(1, "Funkcja wykonana przez dziecko!\n");  
  
    exitcb(); // Powrót do oryginalnego kodu  
  
}
```

```

int main() {

    int pid;

    printf(1, "Proces nadrzędny nie ustawia funkcji\n");

    pid = fork();

    if (pid == 0) {

        printf(1, "Proces podrzędny bez funkcji\n");

        exit();

    }

    wait();

    printf(1, "Proces nadrzędny ustawia funkcję\n");

    forkcb(&testfunc);

    pid = fork();

    if (pid == 0) {

        printf(1, "Proces podrzędny z funkcją\n");

        exit();

    }

    wait();

    printf(1, "Test zakończony\n");

    exit();

}

```

Przydatne linki:

- [książka x86](#)
- [kod źródłowy projektu \(x86\)](#)
- [książka z kodem źródłowym \(x86\)](#)

Lista 8 (Lab) Termin wysłania na SVN oraz oddania do 4.02.2025

1. (5pt) Załóż system plików FAT i VFAT (podobnie jak na wykładzie). Utwórz na nim kilka plików (co najmniej jeden większy od wielkości pojedynczego klastra) i pokaż jak są przechowywane w systemie plików (`hexdump -C` lub `od -Ax -tx1z`) np. jak zmieniła się tablica alokacji (FAT), tablica katalogów (root directory entry) oraz gdzie znajduje się zawartość plików. Usuń wybrany plik i pokaż jak zmienił się system plików. Co należy zrobić aby odzyskać skasowany plik? Wprowadź długą nazwę pliku i pokaż jak jest reprezentowana w systemie FAT i VFAT.
2. (10pt) Załóż system plików dla systemu XV6 (patrz `mkfs`) (podobnie jak na wykładzie). Utwórz na nim kilka plików pod systemem XV6 (co najmniej jeden większy od wielkości pojedynczego bloku) i pokaż jak są przechowywane w systemie plików (`hexdump -C` lub `od -Ax -tx1z` lub `od -Ax -td4z`) np. jak zmienił się obszar i-nodów, gdzie zapisywane są nazwy plików oraz katalogi, gdzie znajduje się zawartość plików. Usuń wybrany plik i pokaż jak zmienił się system plików. Co należy zrobić aby odzyskać skasowany plik, czy jest to możliwe?
3. (20pt) W tym zadaniu zwiększysz maksymalny rozmiar pliku w systemie xv6. Obecnie pliki w xv6 są ograniczone do 140 bloków, czyli 71 680 bajtów. To ograniczenie wynika z faktu, że inode systemu xv6 zawiera 12 "bezpośrednich" (direct) numerów bloków oraz jeden numer bloku "jednokrotnie pośredniego" (single-indirect), który odnosi się do bloku przechowującego do 128 dodatkowych numerów bloków. W sumie daje to $12+128=140$ bloków.
Zmodyfikuj kod systemu plików xv6, aby obsługiwał "dwukrotnie pośredni" (double-indirect) blok w każdym inode, zawierający 128 adresów bloków jednokrotnie pośrednich, z których każdy może zawierać do 128 adresów bloków danych. Dzięki temu plik będzie mógł składać się z maksymalnie $128*128+128+11=16\,523$ bloków (około 8,5 megabajta).
`mkfs` inicjalizuje system plików z mniej niż 1000 wolnymi blokami danych, co jest zbyt małą liczbą, aby pokazać zmiany. Zmień `param.h`, ustawiając `FSSIZE` na:

```
#define FSSIZE      20000 // rozmiar systemu plików w blokach
```

Wykorzystuj plik `big.c` - źródło

```
#include "types.h"
```

```
#include "stat.h"
```

```
#include "user.h"
```

```
#include "fcntl.h"
```

```
int
```

```
main()
```

```
{
```

```
    char buf[512];
```

```
    int fd, i, sectors;
```

```

fd = open("big.file", O_CREATE | O_WRONLY);

if(fd < 0){

    printf(2, "big: cannot open big.file for writing\n");

    exit();

}


sectors = 0;

while(1){

    *(int*)buf = sectors;

    int cc = write(fd, buf, sizeof(buf));

    if(cc <= 0)

        break;

    sectors++;

    if (sectors % 100 == 0)

        printf(2, ".");

}


printf(1, "\nwrote %d sectors\n", sectors);


close(fd);

fd = open("big.file", O_RDONLY);

if(fd < 0){

    printf(2, "big: cannot re-open big.file for reading\n");

    exit();

}

for(i = 0; i < sectors; i++){

    int cc = read(fd, buf, sizeof(buf));

    if(cc <= 0){

        printf(2, "big: read error at sector %d\n", i);
    }
}

```

```

        exit();

    }

    if(*(int*)buf != i){

        printf(2, "big: read the wrong data (%d) for sector %d\n",

            *(int*)buf, i);

        exit();

    }

}

printf(1, "done; ok\n");

exit();

}

```

Dodaj go do listy **UPROGS**, uruchom xv6 i uruchom program **big**. Utworzy on największy plik, jaki xv6 pozwala, i zgłosi wynikowy rozmiar. Powinien wynosić 140 sektorów. Format inode na dysku jest zdefiniowany w strukturze **dinode** w **fs.h**. Zwróć uwagę na pola **NDIRECT**, **NINDIRECT**, **MAXFILE** oraz element **addrs[]** w strukturze **dinode**. Kod, który znajduje dane pliku na dysku, znajduje się w funkcji **bmap()** w **fs.c**. Zrozum, co robi ten kod. **bmap()** jest wywoływany zarówno podczas odczytu, jak i zapisu pliku. Podczas zapisu **bmap()** przydziela nowe bloki, jeśli to konieczne, aby pomieścić zawartość pliku, a także przydziela blok pośredni, jeśli jest potrzebny do przechowywania adresów bloków.

Twoje zadanie

Zmodyfikuj **bmap()**, aby obsługiwał blok dwukrotnie pośredni, oprócz bloków bezpośrednich i bloku jednokrotnie pośredniego. Będziesz musiał zmniejszyć liczbę bloków bezpośrednich do 11 (zamiast 12), aby zrobić miejsce na nowy blok dwukrotnie pośredni. Pierwsze 11 elementów **ip->addrs[]** powinno być blokami bezpośrednimi; 12-ty element powinien być blokiem jednokrotnie pośrednim (tak jak obecnie); 13-ty element powinien być nowym blokiem dwukrotnie pośrednim.

Jeśli implementacja będzie poprawna **big** powinien teraz zgłaszać możliwość zapisania 16 523 sektorów.

Wskazówki

- Zrozum dokładnie funkcję **bmap()**. Powyżej jest narysowany diagram relacji między **ip->addrs[]**, blokiem pośrednim, blokiem dwukrotnie pośrednim, a blokami danych.
- Przemyśl sposób indeksowania bloku dwukrotnie pośredniego i bloków pośrednich, na które wskazuje, za pomocą logicznego numeru bloku.
- Jeśli zmienisz definicję **NDIRECT**, upewnij się, że tworzysz nowy **fs.img**, ponieważ **mkfs** również używa **NDIRECT** do budowy początkowych systemów plików.

Treść zadania bazuje na zadaniu [link](#).

Przydatne linki:

- [książka x86](#)
- [kod źródłowy projektu \(x86\)](#)
- [książka z kodem źródłowym \(x86\)](#)